

RCU (Read-Copy-Update) i odzyskiwanie pamięci w C++20

Wzorzec RCU to technika synchronizacji stworzona dla struktur danych, które są bardzo często czytane, a rzadko modyfikowane (np. tablice routingu, konfiguracje, cache). Główną zaletą RCU jest to, że **wątki czytające nie używają żadnych blokad (muteksów)**, co pozwala im działać z maksymalną prędkością.

W C++20 brakuje standardowej biblioteki rozwiązującej problem zwalniania pamięci po aktualizacji (pojawi się to w C++26). Z tego powodu programiści muszą zaimplementować własny mechanizm śledzenia aktywności wątków, aby wiedzieć, kiedy bezpiecznie wywołać `delete`.

1. Zasada działania RCU

Działanie RCU opiera się na trzech filarach:

1. **Read (Odczyt)**: Wątki czytające sygnalizują "wejście" do sekcji krytycznej odczytu, pobierają wskaźnik i czytają dane.
2. **Copy-Update (Kopia i Aktualizacja)**: Wątek modyfikujący *nie zmienia* oryginalnych danych. Tworzy ich kopię, modyfikuje ją, a następnie używa atomowej instrukcji, aby globalny wskaźnik zaczął wskazywać na nową wersję.
3. **Reclamation (Odzyskiwanie pamięci / Zwalnianie)**: Stary obiekt zostaje zgłoszony do usunięcia (tzw. *Retirement*). System musi poczekać z wykonaniem `delete`, aż minie **Okres Karencji** (*Grace Period*) – czyli moment, gdy wszystkie wątki, które mogły widzieć stary obiekt, zakończą swój odczyt.

2. Implementacja całkowicie bezblokowa (Lock-Free EBR) w C++20

Poniższa implementacja opiera się na algorytmie **Epoch-Based Reclamation (EBR)**. W odróżnieniu od prostszych rozwiązań, ta wersja jest **całkowicie wolna od blokad (lock-free)**. Zamiast wektora i muteksu chroniącego listę obiektów do usunięcia, zastosowano tu bez stos jednokierunkowy (często nazywany stosem Treibera) z użyciem pętli `compare_exchange_weak`.

Aby uniknąć ukrytych alokacji (i potencjalnych blokad) w `std::function`, do zacierania typów (*type erasure*) użyto klasycznego wskaźnika na funkcję.

Kod źródłowy: Lock-Free RCU Domain

```
#include <atomic>
#include <vector>
```

```

#include <iostream>

// Wartość oznaczająca, że wątek aktualnie nie czyta żadnych danych RCU
constexpr uint64_t INACTIVE = static_cast<uint64_t>(-1);

// Struktura węzła dla bezblokowej listy (stosu) obiektów wycofanych.
struct RetiredNode {
    void* ptr;
    uint64_t epoch_when_retired;
    void (*deleter)(void*); // C-style type erasure (brak ukrytych alokacji)
    RetiredNode* next;      // Wskaźnik do następnego węzła w liście lock-free
};

class RCUDomain {
private:
    std::atomic<uint64_t> global_epoch{0};

    // Tablica epok dla poszczególnych wątków.
    std::vector<std::atomic<uint64_t>> thread_epochs;

    // lista (stos) obiektów oczekujących na usunięcie.
    // Brak std::mutex!
    std::atomic<RetiredNode*> retire_list_head{nullptr};

public:
    RCUDomain(int max_threads) : thread_epochs(max_threads) {
        for (auto& te : thread_epochs) {
            te.store(INACTIVE, std::memory_order_relaxed);
        }
    }

    // --- INTERFEJS CZYTELNIKA (Wait-Free) ---

    void read_lock(int thread_id) {
        uint64_t current_epoch = global_epoch.load(std::memory_order_acquire);
        thread_epochs[thread_id].store(current_epoch, std::memory_order_release);
    }

    void read_unlock(int thread_id) {
        thread_epochs[thread_id].store(INACTIVE, std::memory_order_release);
    }

    // --- INTERFEJS MODYFIKATORA / ODZYSKIWANIA (Lock-Free) ---

```

```

template<typename T>
void retire(T* old_ptr) {
    uint64_t current_epoch = global_epoch.load(std::memory_order_acquire);

    // Tworzymy węzeł z informacją o usuwanym obiekcie
    RetiredNode* node = new RetiredNode{
        old_ptr,
        current_epoch,
        [](void* p){ delete static_cast<T*>(p); }, // Statyczna funkcja zwalniania
        nullptr
    };

    // LOCK-FREE PUSH: Wstawiamy węzeł na początek listy (algorytm Treiber Stack)
    RetiredNode* expected = retire_list_head.load(std::memory_order_relaxed);
    do {
        node->next = expected;
        // compare_exchange_weak atomowo sprawdza czy głowa to nadal 'expected'.
        // Jeśli tak -> nadpisuje ją 'node' i zwraca true.
        // Jeśli nie -> aktualizuje 'expected' nową wartością głowy i wraca do pętli.
    } while (!retire_list_head.compare_exchange_weak(
        expected, node, std::memory_order_release, std::memory_order_relaxed));

    // Zwiększamy epokę.
    global_epoch.fetch_add(1, std::memory_order_acq_rel);

    // Próba odzyskania pamięci
    reclaim();
}

```

private:

```

void reclaim() {
    // KROK 1: Wyznacz "najstarszą aktywną epokę"
    uint64_t min_active_epoch = global_epoch.load(std::memory_order_acquire);
    for (const auto& te : thread_epochs) {
        uint64_t thread_e = te.load(std::memory_order_acquire);
        if (thread_e != INACTIVE && thread_e < min_active_epoch) {
            min_active_epoch = thread_e;
        }
    }
}

```

// KROK 2: Atomowe "ukradzenie" całej listy odzyskiwania
// To jest klucz do lock-free rekuperacji. Pobieramy całą listę

```

// i podmieniamy globalną głowę na nullptr, wszystko w jednej atomowej instrukcji.
RetiredNode* current_list = retire_list_head.exchange(nullptr, std::memory_order_acquire);

// Lokalne listy dla obiektów, które posortujemy
RetiredNode* to_delete = nullptr;
RetiredNode* to_keep = nullptr;

// Rozdzielamy obiekty na te do usunięcia i te do zachowania na później
while (current_list != nullptr) {
    RetiredNode* node = current_list;
    current_list = current_list->next;

    if (node->epoch_when_retired < min_active_epoch) {
        // Bezpieczne - Okres karencji minął
        node->next = to_delete;
        to_delete = node;
    } else {
        // Niebezpieczne - Ktoś może jeszcze czytać
        node->next = to_keep;
        to_keep = node;
    }
}

// KROK 3: Fizyczne usunięcie bezpiecznych obiektów
while (to_delete != nullptr) {
    RetiredNode* node = to_delete;
    to_delete = to_delete->next;
    node->deleter(node->ptr); // Usuwa właściwy obiekt (np. Config)
    delete node;           // Usuwa węzeł struktury
}

// KROK 4: Lock-free włożenie z powrotem obiektów, które muszą jeszcze poczekać
while (to_keep != nullptr) {
    RetiredNode* node = to_keep;
    to_keep = to_keep->next;

    RetiredNode* expected = retire_list_head.load(std::memory_order_relaxed);
    do {
        node->next = expected;
    } while (!retire_list_head.compare_exchange_weak(
        expected, node, std::memory_order_release, std::memory_order_relaxed));
}
}

```

```
};
```

3. Przykładowe użycie w kodzie z danymi

Integracja odbywa się poprzez `std::atomic<T*>`. Wątki czytające i modyfikujące działają teraz bez zatrzymywania się na blokadach (muteksach) w żadnym momencie swojego życia.

```
struct Config { int parameter_a; double parameter_b; };
std::atomic<Config*> global_config{new Config{10, 3.14}};
RCUDomain rcu(4);

// --- WĄTEK CZYTAJĄCY ---
void reader_thread(int my_tid) {
    rcu.read_lock(my_tid); // Całkowicie bez blokad

    // Atomowe pobranie z barierą "acquire"
    Config* cfg = global_config.load(std::memory_order_acquire);

    std::cout << "Czytam konfigurację: " << cfg->parameter_a << std::endl;

    rcu.read_unlock(my_tid);
}

// --- WĄTEK MODYFIKUJĄCY ---
void updater_thread() {
    Config* new_cfg = new Config{20, 6.28};

    // Publikacja nowego wskaźnika z barierą "release"
    Config* old_cfg = global_config.exchange(new_cfg, std::memory_order_acq_rel);

    // Zgłoszenie do kolejki bez blokad
    rcu.retire(old_cfg);
}
```

4. Pamięć, bariery i memory_order (Acquire vs Release)

W wielowątkowym C++ sam fakt użycia typu atomowego do wskaźnika nie wystarczy, by kod

był poprawny. Trzeba narzucić kompilatorowi i procesorowi odpowiedni **rygor pamięciowy** (*memory ordering*). Klasykami są tu `acquire` oraz `release`.

Wyobraź sobie następującą sytuację w wątku modyfikującym:

1. Alokacja pamięci dla `new_cfg`.
2. Zapis wartości 20 do `parameter_a`.
3. Zapis wartości 6.28 do `parameter_b`.
4. Publikacja wskaźnika: `global_config.store(new_cfg)`.

Problem braku barier (Zagrożenie reorderingiem)

Współczesne procesory oraz kompilatory drastycznie optymalizują kod, zmieniając kolejność wykonywanych instrukcji (tzw. *reordering*), o ile nie zmienia to logiki dla *danego* wątku.

Gdybyśmy użyli luźnej bariery (`std::memory_order_relaxed`), procesor mógłby uznać: *"Zapisanie adresu do wskaźnika jest szybkie. Zrobię krok 4 przed krokiem 2 i 3"*.

W efekcie nowy adres wskaźnika zostałby opublikowany, inny wątek zacząłby go czytać, ale pod tym adresem **nie byłoby jeszcze prawidłowych danych** (dane zaktualizowałyby się mikrosekundę później). Program by się zawiesił.

Rozwiązanie: Release i Acquire

Aby temu zapobiec, stosujemy relację **Acquire-Release**, która przypomina "zapięczętowanie" paczki przed wysyłką i upewnienie się, że paczka dotarła w całości przed jej otwarciem.

- **`std::memory_order_release` (Używane przez Publikującego):**
Mówi procesorowi/kompilatorowi: *"Wszystkie operacje zapisu do pamięci, które wykonałem przed tą instrukcją (np. inicjalizacja pól obiektu `new_cfg`), muszą być w pełni widoczne w pamięci GŁÓWNEJ zanim opublikujesz ten nowy wskaźnik"*. Stanowi to barierę, przez którą instrukcje sprzed zapisu wskaźnika nie mogą "przeskoczyć" w dół.
- **`std::memory_order_acquire` (Używane przez Czytelnika):**
Mówi: *"Jeżeli właśnie odczytałem ten wskaźnik, zagwarantuj mi, że żadne moje późniejsze instrukcje czytające jego zawartość (np. `cfg->parameter_a`) nie wykonają się, zanim nie upewnię się, że pobrałem najświeższą, pełną strukturę danych"*. Bariera ta zapobiega "przeskoczeniu" instrukcji odczytu powyżej pobrania wskaźnika.

5. Alternatywne podejście: Interval-Based Reclamation (IBR)

Oprócz Epoch-Based Reclamation (EBR) przedstawionego w rozdziale 2, inną zaawansowaną techniką bezblokowego odzyskiwania pamięci dla RCU jest **Interval-Based Reclamation (IBR)**. Została ona zaprojektowana, aby zminimalizować wady EBR związane z blokowaniem mechanizmu czyszczącego.

Problem z epokami (EBR)

W EBR proces usuwania pamięci zależy od wyznaczenia "najstarszej aktywnej epoki". Jeżeli jeden z wątków czytających zostanie wstrzymany przez system operacyjny na bardzo długi czas będąc w trakcie czytania (np. podczas IO lub intensywnych obliczeń), epoka dla tego wątku nie zostanie zaktualizowana. W efekcie, **żaden wycofany obiekt nie będzie mógł zostać usunięty**, dopóki ów wątek nie zakończy pracy, co grozi przepełnieniem pamięci.

Zasada działania Interwałów (IBR)

IBR rozwiązuje ten problem śledząc precyzyjne "okna czasu" (interwały) życia obiektu, a nie zgrubne "epoki".

1. **Zegar Globalny:** Zamiast epoki zwiększanej losowo, IBR używa ściśle rosnącego globalnego zegara operacji (licznika zdarzeń).
2. **Narodziny i Śmierć (Wycofanie):** Kiedy obiekt jest wycofywany (retire), system wykonuje na zegarze instrukcję `fetch_add(1)` i taguje ten obiekt unikalnym momentem jego usunięcia (np. obiekt X został usunięty w czasie $T = 100$).
3. **Czas wejścia wątku:** Kiedy wątek zaczyna czytać, kopiuje do swojej zmiennej lokalnej aktualny stan globalnego zegara jako swój "czas wejścia" (np. zacząłem czytać w momencie $T = 105$).
4. **Warunek usunięcia:** Obiekt X wycofany w czasie $T = 100$ jest bezpieczny do fizycznego usunięcia (delete) tylko wtedy, gdy żaden aktywny wątek nie rozpoczął pracy *wcześniej* lub *w tym samym* momencie. Innymi słowy, sprawdzamy czy dla wszystkich aktywnych wątków czytających zachodzi: `thread_start_time > 100`.

Kompletna implementacja IBR (C++20)

Poniższy kod przedstawia w pełni funkcjonalną, bezblokadową wersję mechanizmu IBR:

```
#include <atomic>
#include <vector>

constexpr uint64_t IBR_INACTIVE = static_cast<uint64_t>(-1);

struct IBRNode {
    void* ptr;
    uint64_t retire_time; // Zapis momentu usunięcia obiektu
    void (*deleter)(void*);
    IBRNode* next;
};

class IBRDomain {
private:
    std::atomic<uint64_t> global_clock{0};
```

```
// Czas rozpoczęcia odczytu dla każdego wątku
std::vector<std::atomic<uint64_t>> thread_start_times;
std::atomic<IBRNode*> retire_list_head{nullptr};
```

public:

```
IBRDomain(int max_threads) : thread_start_times(max_threads) {
    for (auto& tst : thread_start_times) {
        tst.store(IBR_INACTIVE, std::memory_order_relaxed);
    }
}
```

```
void read_lock(int thread_id) {
    // Wątek "stempluje" swój czas wejścia przed odczytem struktury RCU
    uint64_t current_time = global_clock.load(std::memory_order_acquire);
    thread_start_times[thread_id].store(current_time, std::memory_order_release);
}
```

```
void read_unlock(int thread_id) {
    // Ważne: Zwolnienie blokady, aby wątek nie opóźniał odzyskiwania pamięci!
    thread_start_times[thread_id].store(IBR_INACTIVE, std::memory_order_release);
}
```

template<typename T>

```
void retire(T* old_ptr) {
    // Inkrementujemy zegar i uzyskujemy dokładny czas wycofania
    uint64_t r_time = global_clock.fetch_add(1, std::memory_order_acq_rel);
```

```
    IBRNode* node = new IBRNode{
        old_ptr,
        r_time,
        [](void* p){ delete static_cast<T*>(p); },
        nullptr
    };
}
```

// Lock-free push na stos wskaźników

```
IBRNode* expected = retire_list_head.load(std::memory_order_relaxed);
do {
    node->next = expected;
} while (!retire_list_head.compare_exchange_weak(
    expected, node, std::memory_order_release, std::memory_order_relaxed));
```

reclaim(); // Uruchomienie próby czyszczenia

```
}
```

private:

```
void reclaim() {
    // KROK 1: Wyznacz najstarszy czas rozpoczęcia operacji przez dowolny wątek
    uint64_t min_active_time = global_clock.load(std::memory_order_acquire);
    for (const auto& tst : thread_start_times) {
        uint64_t start_time = tst.load(std::memory_order_acquire);
        if (start_time != IBR_INACTIVE && start_time < min_active_time) {
            min_active_time = start_time;
        }
    }
}

// KROK 2: Atomowe "ukradzenie" całej listy odzyskiwania
IBRNode* current_list = retire_list_head.exchange(nullptr, std::memory_order_acquire);

IBRNode* to_delete = nullptr;
IBRNode* to_keep = nullptr;

// KROK 3: Klasyfikacja obiektów (Bezpieczne vs Niebezpieczne)
while (current_list != nullptr) {
    IBRNode* node = current_list;
    current_list = current_list->next;

    // KLUCZOWA RÓŻNICA WZGLĘDEM EBR:
    // Jeśli obiekt został wycofany WCZEŚNIEJ niż zaczął pracę jakkolwiek
    // z obecnie działających wątków, to na 100% nikt go już nie widzi.
    if (node->retire_time < min_active_time) {
        node->next = to_delete;
        to_delete = node;
    } else {
        node->next = to_keep;
        to_keep = node;
    }
}

// KROK 4: Usuwanie pamięci
while (to_delete != nullptr) {
    IBRNode* node = to_delete;
    to_delete = to_delete->next;
    node->deleter(node->ptr);
    delete node;
}
```

```

// KROK 5: Lock-free włożenie z powrotem niebezpiecznych obiektów
while (to_keep != nullptr) {
    IBRNode* node = to_keep;
    to_keep = to_keep->next;

    IBRNode* expected = retire_list_head.load(std::memory_order_relaxed);
    do {
        node->next = expected;
    } while (!retire_list_head.compare_exchange_weak(
        expected, node, std::memory_order_release, std::memory_order_relaxed));
}
};

```

Co wybrać: EBR czy IBR?

- **Wybierz EBR**, jeśli zależy Ci na absolutnej maksymalizacji prędkości zapisu i usuwania. W EBR `retire()` wykonuje zazwyczaj zwykłe odczyty, podczas gdy w IBR operacja `fetch_add(1)` staje się "wąskim gardłem" (bottleneck), gdy wiele wątków masowo usuwa obiekty w tym samym czasie.
- **Wybierz IBR**, jeśli Twój system musi być odporny na asymetrię wątków – np. gdy niektóre wątki wykonują bardzo długie i skomplikowane operacje odczytu (co wstrzymałoby globalną epokę), a inne w tym czasie muszą uwalniać pamięć, by nie doprowadzić do problemów wydajnościowych. Zaletą implementacji IBR jest dokładniejsze izolowanie problemu wolnych czytelników do najpóźniejszych operacji.